

Neural Networks — Key Concepts

Lesson 9 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

20 November 2026 · a one-page index

Contents

One unit, and what it cannot do	1
Running the network	2
Backpropagation	2
More than two classes	2
Why deep networks fail to train	2
Initialisation	3
Training in practice	3
Regularisation	3
How much network	3

An index, not a summary. Each entry says what a term means and where the handout treats it properly. Use this to find your way back into the text, or to check before an exercise that there is no word here you could not define.

One unit, and what it cannot do

Perceptron. The ancestor: weights, a bias, and a hard threshold. Replace the threshold with a sigmoid and it is Lesson 4's logistic regression unit. → § 2.1

A neuron is a line. The set of inputs a unit is undecided about is a hyperplane. Everything one unit can express is “which side are you on, and how far”. → § 2.1

Hidden layer. A layer whose outputs feed another layer rather than the answer. → § 3

Representation learning. What a hidden layer actually does: it moves the data until the last layer's single line is enough. It classifies nothing. → § 3.3

Capacity against findability. A solution existing and gradient descent finding it are different things. Networks are built wider than the task needs because extra units are extra starting points. → § 3.4

Universal approximation. One hidden layer with enough units can approximate any continuous function on a bounded region. It says such a network *exists* — nothing about how many units, or whether training finds it. → § 3.5

Running the network

Forward propagation. A weight matrix, a bias, an activation — once per layer. → § 4.1

Why the shapes matter. This course puts examples in rows; many textbooks use columns, and every transpose flips. Mixing the two conventions mid-derivation is the commonest way to produce algebra that looks right. → § 4.1

Why examples never mix. The row index passes through every operation untouched, which is what makes mini-batching valid. → § 4.1

Backpropagation

The idea. The repeated sub-expressions of the chain rule, computed once each from right to left, at about the cost of one extra forward pass. → § 5.1

The backpropagated signal. The cost's derivative with respect to a layer's pre-activations. Given it for a layer you get that layer's weight gradients *and* the same quantity for the one below. → § 5.1

Why sigmoid and cross-entropy belong together. The sigmoid's derivative cancels exactly against the logarithm's, leaving prediction minus truth — so a confidently wrong example gets a large gradient instead of a vanishing one. → § 5.2

Memory cost. Every layer's activations must be kept from the forward pass, so memory grows with depth times batch size. That, not arithmetic, usually limits batch size. → § 5.4

Gradient checking. Comparing the analytic gradient against a central finite difference. Four lines, and the only defence against a wrong gradient that trains anyway. → § 5.5

More than two classes

Softmax. One score per class, exponentiated and normalised; two classes give back the sigmoid. → § 6.1

The same cancellation. Softmax with categorical cross-entropy also collapses to prediction minus truth, for the same reason and not by coincidence. → § 6.2

Why deep networks fail to train

Why a non-linearity is needed. Without one, any number of layers composes into a single linear layer, and depth buys nothing. → § 7.1

Vanishing gradient. The activation's derivative multiplies the backpropagated signal once per layer. If it is bounded below one, the product shrinks geometrically with depth. → § 7.2

The sigmoid's bound. Its derivative never exceeds one quarter, so a sigmoid layer divides the gradient by about four. This is the number to carry out of the lesson. → § 7.2

Rectified linear unit (ReLU). An activation that is the identity on the positive side, so its derivative there is exactly one and it does not saturate. → § 7.2

Dead ReLU. A unit whose pre-activation is negative for every example: it outputs zero, receives zero gradient, and never recovers. Wasted capacity rather than catastrophe, which is why it is easy to miss. → § 7.4

Initialisation

Why not zero. Symmetry is the usual explanation, but all-zero weights do something stronger: every gradient is exactly zero and the whole network is frozen but for one bias. → § 8.1

Why not large. With a bounded activation it does not explode, it *saturates* — and a saturated layer passes signal forward and nothing backward. → § 8.2

Glorot and He. Set the weight variance to one over the number of inputs, or two over it for a rectified linear unit, so the signal’s variance survives depth instead of compounding. → § 8.2

Propagating the variance. Independent terms add their variances, and a zero-mean weight independent of its input contributes $\text{Var}(w) \mathbb{E}[a^2]$ — the two steps that make $n \text{Var}(w)$ the factor per layer. → § 8.2

Training in practice

The learning rate. The first thing to sweep and the last to guess. Too small looks exactly like a network that is too small — and adding units makes it slower, not better. → § 9.1

Mini-batch gradient descent. A few dozen to a few hundred examples per step: the estimate is noisy but unbiased, the arithmetic vectorises, the noise helps escape shallow minima. → § 9.2

Momentum. A running average of past gradients, so consistent directions accumulate and oscillating ones cancel. The direct repair for a stretched valley. → § 9.3

Adam. Momentum plus a per-parameter step size, which makes it forgiving of a badly chosen global learning rate. Its value here was removing the bad case, not raising the mean. → § 9.3

Regularisation

Memorising and generalising at once. A network with far more parameters than examples can fit the training set perfectly and still generalise — which classical bias-variance reasoning does not predict, and which is an open question. → § 10.1

Loss against accuracy. Validation loss turns upward while accuracy stays flat: the network is not getting more answers wrong, it is getting more confident about the ones it already has wrong. Stop on the loss. → § 10.1

Early stopping. Keep the weights from the best epoch: the cheapest regulariser there is. → § 10.2

Weight decay (L_2). Add a charge for weight size, so each step shrinks the weight slightly before the data moves it. Identical in form to Lesson 3’s Ridge. → § 10.2

Dropout. Delete each unit independently on every training batch and rescale the survivors, so no unit can rely on any particular other one being present. → § 10.2

Report differences against their spread. Three regularisers separated by less than two standard deviations cannot be ranked by one run, however tidy the table looks. → § 10.3

How much network

Width against optimisation. Past a few units the lines are available and gradient descent stops putting them to work — the binding constraint moves from capacity to the optimiser. → § 11.2

The measuring instrument's ceiling. When labels are recorded imperfectly, measured accuracy blends true agreement with the recorder's error rate; only the lower number is observable. → § 11.3